



Protocol Audit Report

Version 1.0

Cyfrin.io

September 18, 2025

Protocol Audit Report

thecil

Sep 18, 2025

Prepared by: Cyfrin Lead Auditors: - thecil

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can [deposit](#) assets into [ThunderLoan](#) and be given [AssetTokens](#) in return. These [AssetTokens](#) gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, we're using the famous on-chain TSwap price oracle.

Disclaimer

The thecil team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 8803f851f6b37e99eab2e94b4690c8b70e26b3f6
- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- ERC20s:
 - USDC
 - DAI
 - LINK
 - WETH

Scope

```
1  |-- interfaces
2  |   |-- IFlashLoanReceiver.sol
3  |   |-- IPoolFactory.sol
4  |   |-- ITSwapPool.sol
5  |   |-- IThunderLoan.sol
6  |-- protocol
7  |   |-- AssetToken.sol
8  |   |-- OracleUpgradeable.sol
9  |   |-- ThunderLoan.sol
10 |-- upgradedProtocol
11   |-- ThunderLoanUpgraded.sol
```

Lines of Code

Filepath	nSLOC
src/interfaces/IFlashLoanReceiver.sol	13
src/interfaces/IPoolFactory.sol	4
src/interfaces/ITSwapPool.sol	4
src/interfaces/IThunderLoan.sol	4
src/protocol/AssetToken.sol	65
src/protocol/OracleUpgradeable.sol	27
src/protocol/ThunderLoan.sol	225

Filepath	nSLOC
src/upgradedProtocol/ThunderLoanUpgraded.sol	221
Total	563

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

Issues found

Severity	Number of issues found
High	3
Medium	1
Low	3
Info	2
Gas	1
Total	10

Findings

High

[H-1] - Erroneous ThunderLoan::updateExchangeRate in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly set the exchange rate.

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between `assetTokens` and underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function, updates this rate without collecting any fees, by doing this, the `redeem` function will think it has more fees than it really does, which blocks redemption and incorrectly set the exchange rate.

```
1      function deposit(IERC20 token, uint256 amount) external
2          revertIfZero(amount) revertIfNotAllowedToken(token) {
3          AssetToken assetToken = s_tokenToAssetToken[token];
4          uint256 exchangeRate = assetToken.getExchangeRate();
5          uint256 mintAmount = (amount * assetToken.
6              EXCHANGE_RATE_PRECISION()) / exchangeRate;
7          emit Deposit(msg.sender, token, amount);
8          assetToken.mint(msg.sender, mintAmount);
9          @> uint256 calculatedFee = getCalculatedFee(token, amount);
10         @> assetToken.updateExchangeRate(calculatedFee);
11         token.safeTransferFrom(msg.sender, address(assetToken), amount)
12         ;
13     }
```

Impact: There are several impacts to this bug.

1. The `redeem` function is blocked because the protocol thinks the owed tokens is more than it has, preventing users from redeeming their tokens.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than deserved.

Proof of Concept:

1. LP deposits.
2. User takes a flashloan.
3. It is now impossible for LP to redeem.

Proof of Code

Place the following into `ThunderLoanTest.t.sol` and execute the unit test.

```
1      function test_redeemAfterLoan() public setAllowedToken hasDeposits
2      {
3          // copy/paste from testFlashLoan()
4          uint256 amountToBorrow = AMOUNT * 10;
5          uint256 calculatedFee = thunderLoan.getCalculatedFee(
6              tokenA,
7              amountToBorrow
8          );
9          vm.startPrank(user);
10         tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
11         thunderLoan.flashloan(
12             address(mockFlashLoanReceiver),
13             tokenA,
14             amountToBorrow,
15             ""
16         );
17         vm.stopPrank();
18
19         uint256 amountToRedeem = type(uint256).max;
20         vm.startPrank(LiquidityProvider);
21         thunderLoan.redeem(tokenA, amountToRedeem);
22         vm.stopPrank();
23     }
```

Recommended Mitigation: Remove the incorrectly updated exchange rate lines from the `deposit` function.

```
1      function deposit(IERC20 token, uint256 amount) external
2      revertIfZero(amount) revertIfNotAllowedToken(token) {
3          AssetToken assetToken = s_tokenToAssetToken[token];
4          uint256 exchangeRate = assetToken.getExchangeRate();
5          uint256 mintAmount = (amount * assetToken.
6              EXCHANGE_RATE_PRECISION()) / exchangeRate;
7          emit Deposit(msg.sender, token, amount);
8          assetToken.mint(msg.sender, mintAmount);
9          - uint256 calculatedFee = getCalculatedFee(token, amount);
10         - assetToken.updateExchangeRate(calculatedFee);
11         token.safeTransferFrom(msg.sender, address(assetToken), amount)
12         ;
13     }
```

[H-2] - All the funds can be stolen if the flash loan is returned using deposit()

Description: An attacker can acquire a flash loan and deposit funds directly into the contract using the `deposit()`, enabling stealing all the funds.

Impact: The `flashloan()` performs a crucial balance check to ensure that the ending balance, after the

flash loan, exceeds the initial balance, accounting for any borrower fees. This verification is achieved by comparing endingBalance with startingBalance + fee. However, a vulnerability emerges when calculating endingBalance using token.balanceOf(address(assetToken)).

Exploiting this vulnerability, an attacker can return the flash loan using the deposit() instead of repay(). This action allows the attacker to mint AssetToken and subsequently redeem it using redeem(). What makes this possible is the apparent increase in the Asset contract's balance, even though it resulted from the use of the incorrect function. Consequently, the flash loan doesn't trigger a revert.

Proof of Concept:

Check the unit test `ThunderLoanTest::test_useDepositInstedOfRepayToStealFunds` as proof of code for this bug.

Recommended Mitigation: Add a check in deposit() to make it impossible to use it in the same block of the flash loan. For example registering the block.number in a variable in flashloan() and checking it in deposit().

[H-3] - Storage Collision between ThunderLoan and ThunderLoanUpgraded.

Description: The `ThunderLoanUpgraded` contract is a version of the `ThunderLoan` contract that has a different implementation of the storage slots.

In `ThunderLoan` contract, the storage slot 2 contains the variable `s_feePrecision` which is used for calculating the fee. While in `ThunderLoanUpgraded` contract, the storage slot 2 contains the variable `s_flashLoanFee` which is used to calculate the flash loan fee.

The `s_feePrecision` from the `thunderloan.sol` was changed to `uint256 constant FEE_PRECISION = 1e18` variable which will no longer be assessed from the state variable.

Impact: This will cause the location at which the upgraded version will be pointing to for some significant state variables like `s_flashLoanFee` to be wrong because `s_flashLoanFee` is now pointing to the slot of the `s_feePrecision` in the `thunderloan.sol` and when this fee is used to compute the fee for flashloan it will return a fee amount greater than the intention of the developer. `s_currentlyFlashLoaning` might not really be affected as it is back to default when a flashloan is completed but still to be noted that the value at that slot can be cleared to be on a safer side.

Proof of Concept: (Proof of Code)

This is the actual storage for each `ThunderLoan` and `ThunderLoanUpgraded` contract

Verify by executing the following commands for each

```
1 forge inspect ThunderLoan storage
2 forge inspect ThunderLoanUpgraded storage
```


ThunderLoan storage:

Name	Type	Slot	Offset	Bytes
s_poolFactory	address	0	0	20
s_tokenToAssetToken	mapping(IERC20 => AssetToken)	1	0	32
s_feePrecision	uint256	2	0	32
s_flashLoanFee	uint256	3	0	32
s_currentlyFlashLoaning	mapping(IERC20 => bool)	4	0	32

ThunderLoanUpgraded storage:

Name	Type	Slot	Offset	Bytes
s_poolFactory	address	0	0	20
s_tokenToAssetToken	mapping(IERC20 => AssetToken)	1	0	32
s_flashLoanFee	uint256	2	0	32
s_currentlyFlashLoaning	mapping(IERC20 => bool)	3	0	32

Recommended Mitigation: The team should should make sure the the fee is pointing to the correct location as intended by the developer.

Medium

[M-2] - Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reservers are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providint liquidity.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from `ThunderLoan` from 1000 `tokenA`. They are charged the original fee `fee1`. During the flash loan, they do the following:
 1. User sells 1000 `tokenA`, tanking the price.
 2. Instead of repaying right away, the user takes out another flash loan for another 1000 `tokenA`.
 1. Due the fact that the way `ThunderLoan` calculates price based on the `TSwapPool` this second flash loan is substantially cheaper.

```
1     function getPriceInWeth(address token) public view returns (uint256
2         ) {
3         address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(
4             token);
5         return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth
6             ();
7     }
```

- ```
1 3. The user then repays the first flash loan, and then repays the
 second flash loan.
```

Check the unit test `ThunderLoanTest::test_oracleManipulation` as proof of code for this bug.

**Recommended Mitigation:** Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

## Low

### [L-1] - Unused Errors

**Description:** The `ThunderLoan` & `ThunderLoanUpgraded` contracts contains declared errors, yet no function ever reverts with them. Dead declarations bloat bytecode, increase deployment costs, and distract maintainers who must verify their relevance.

**Impact:** Low.

**Proof of Concept:**

The following contracts have unused errors:

- `ThunderLoan` contains `ThunderLoan__ExchangeRateCanOnlyIncrease` unused error.
- `ThunderLoanUpgraded` contains `ThunderLoan__ExchangeRateCanOnlyIncrease` unused error.

**Recommended Mitigation:** We recommend either removing these errors if they are unnecessary, or using them in the appropriate places where the corresponding checks are relevant.

## [L-2] - Initializer Can Be Front-Run to Gain Control of the Program

**Description:** The `initialize` function in the `ThunderLoan` & `ThunderLoanUpgraded` contracts allows anyone to call it with any address as the `tswapAddress`. This could be exploited by an attacker to gain control of the program. The `msg.sender` should be restricted to only the contract owner or a trusted party.

**Impact:** Low.

### Proof of Concept:

The actual codebase for the `initialize` function is as follows:

```
1 function initialize(address tswapAddress) external initializer {
2 __Ownable_init(msg.sender);
3 __UUPSUpgradeable_init();
4 __Oracle_init(tswapAddress);
5 s_feePrecision = 1e18;
6 s_flashLoanFee = 3e15; // 0.3% ETH fee
7 }
```

There is no verification on the `tswapAddress` and there is no restrictions on the `msg.sender`, which means that anyone can call this function and gain control of the program.

### Recommended Mitigation:

1. Make sure the deployment and initialization of the program occur in the same transaction.
2. Add a modifier such as `onlyOwner` to restrict access to the `initialize` function. This will prevent unauthorized calls and ensure that only authorized parties can control the program.

## [L-3] - Missing NatSpec Comments

**Description:** All the contracts in the code-base are missing or have incomplete code documentation, which affects the understandability, auditability, and usability of the code. Solidity contracts can use a special form of comments to provide rich documentation for functions, return variables, parameters, etc. This special form is named the Ethereum Natural Language Specification Format (NatSpec).

**Impact:** Understandability, auditability, and usability of the code are affected.

**Proof of Concept:**

The following functions are missing NatSpec comments:

- `ThunderLoan::deposit.`
- `ThunderLoan::flashloan.`
- `ThunderLoan::repay.`
- `ThunderLoan::setAllowedToken.`
- `ThunderLoan::getCalculatedFee.`
- `ThunderLoanUpgraded::deposit.`
- `ThunderLoanUpgraded::flashloan.`
- `ThunderLoanUpgraded::repay.`
- `ThunderLoanUpgraded::setAllowedToken.`
- `ThunderLoanUpgraded::getCalculatedFee.`

**Recommended Mitigation:** Consider adding in full NatSpec comments for all functions to have complete code documentation for future use.

## Informational

**[I-1] - `ThunderLoan::initialize` function parameter `tswapAddress` should be renamed to `poolFactoryAddress` to match the name of the `OracleUpgradeable` contract.**

**Description:** The `initialize` function parameter `tswapAddress` should be renamed to `poolFactoryAddress` to match the name of the `OracleUpgradeable` contract. This change will make the contract more consistent with other contracts in the protocol.

**Impact:** Low.

**Proof of Concept:**

In `ThunderLoan.sol`:

```
1 function initialize(address tswapAddress) external initializer {
2 __Ownable_init(msg.sender);
3 __UUPSUpgradeable_init();
4 __Oracle_init(tswapAddress);
5 s_feePrecision = 1e18;
6 s_flashLoanFee = 3e15; // 0.3% ETH fee
7 }
```

In `OracleUpgradeable.sol`:

```
1 function __Oracle_init(address poolFactoryAddress) internal
 onlyInitializing {
2 __Oracle_init_unchained(poolFactoryAddress);
3 }
```

**Recommended Mitigation:** Rename the `tswapAddress` parameter to `poolFactoryAddress` in order to harmonize the variables names across the protocol.

```
1 - function initialize(address tswapAddress) external initializer {
2 + function initialize(address poolFactoryAddress) external
 initializer {
3 __Ownable_init(msg.sender);
4 __UUPSUpgradeable_init();
5 - __Oracle_init(tswapAddress);
6 + __Oracle_init(poolFactoryAddress);
7 s_feePrecision = 1e18;
8 s_flashLoanFee = 3e15; // 0.3% ETH fee
9 }
```

## [I-2] - Consider making `public` functions `external`.

**Description:** Several functions in the `ThunderLoan` and `ThunderLoanUpgraded` contracts that are marked as `public` could be declared as `external` to save gas costs and improve performance. It is particularly relevant for functions that are not called internally within the contract.

**Impact:** Low.

### Proof of Concept:

The following functions are `public`:

- `ThunderLoan::repay`.
- `ThunderLoan::getAssetFromToken`.
- `ThunderLoan::isCurrentlyFlashLoaning`.
- `ThunderLoanUpgraded::repay`.
- `ThunderLoanUpgraded::getAssetFromToken`.
- `ThunderLoanUpgraded::isCurrentlyFlashLoaning`.

**Recommended Mitigation:** We recommend changing the visibility of the following functions from `public` to `external` where appropriate.

## Gas

### [G-1] - ThunderLoan::s\_feePrecision can be declared as constant or immutable

**Description:** The `s_feePrecision` variable in the `ThunderLoan` contract is not designed to be modified after deployment, as no setter function exists. Keeping it mutable introduces unnecessary complexity, gas cost, and potential misuse.

**Impact:** Low.

**Proof of Concept:**

This is the actual implementation for the `ThunderLoan::s_feePrecision` variable:

In `ThunderLoan.sol`:

```
1 uint256 private s_feePrecision;
2
3 function initialize(address tswapAddress) external initializer {
4 __Ownable_init(msg.sender);
5 __UUPSUpgradeable_init();
6 __Oracle_init(tswapAddress);
7 s_feePrecision = 1e18;
8 s_flashLoanFee = 3e15; // 0.3% ETH fee
9 }
```

The variable is set to `1e18`, which is a common precision for financial calculations. This value can be used throughout the contract without needing to change it.

There is no setter function for `s_feePrecision`, which means it cannot be changed after the initialization.

**Recommended Mitigation:** Consider marking `s_feePrecision` as constant or immutable to enforce immutability and improve gas efficiency.

PD: Consider the issue at [H-3] – `Storage Collision between ThunderLoan and ThunderLoanUpgraded` to avoid creating any bug related to.